

INSTITUTO SUPERIOR DOM BOSCO

Departamento: TICs

CURSO: Licenciatura em Ensino de Programação informática

3º Ano

Modulo: Engenharia de Software

Tema:

Formando:
Oslerson Muveca

Formador:
MSc. Romário Tomo

Maputo, Outubro de 2026

Ìndice

1 Capitulo I: Descrição do Projecto 6

1.1 Introdução 6

1.2 Problematização 6

1.3 Justificativa 7

1.4 Objectivos 7

1.4.1 Objectivo geral 7

1.4.2 Objectivos específicos 7

2 Capitulo II: Especificação de requisitos 9

2.1 Actores do Sistema 9

2.2 Funcionalidades Principais do Sistema 9

2.3 Requisitos funcionais 10

2.4 Requisitos Não Funcionais 12

2.5 Regras de negócio 13

3 Capitulo III: Arquitectura do Sistema 14

3.1 Modelo de Desenvolvimento 14

3.2 Arquitectura do Sistema 15

3.3 Atributos de Qualidade 16

3.3.1 Segurança 16

3.3.2 Usabilidade 17

3.3.3 Desempenho 17

3.3.4 Integração de dados 17

3.3.5 Manutenibilidade 17

3.4 Linguagens e Tecnologias 17

3.5 Modelo Entidade – Relacionamento (MER) 19

3.6 Identificação das entidades 21

3.6.1 Atributos das entidades 21

3.7 Relacionamentos e cardinalidades 23

3.7.1 Utilizador – Objecto 23

3.7.2 Utilizador – PedidoRecuperacao 23

3.7.3 Objecto – PedidoRecuperacao 23

4 Capitulo IV: Desenho e Implementação 24

4.1 Diagrama de caso de uso 24

4.1.1 Especificação dos casos de uso 26

4.2 Diagrama de Actividades 31

4.3 Diagrama de Classes 33

4.4 Diagrama de Sequência 34

4.5 Protótipo das Interfaces 37

5 Capitulo V: Padrões de projecto 49

5.1 Padrões de projecto 49

5.1.1 Padrão singleton 49

5.1.2 Padrão Model-View-Controller (MVC) 51

6 Capitulo VI: Especificação Formal 53

7 Capitulo VII: Testes 55

7.1 Testes de software 55

7.2 Teste Unitário 55

7.3 Testes de integração 55

7.4 Aplicação dos testes 55

8 Capitulo VIII: Qualidade e Melhorias 56

8.1 Qualidade de Projecto 56

8.2 Métricas 56

9 Capitulo Ix: Gestão de Versões 58

9.1 Controle de versões 58

10 Capitulo IX: Conclusão 59

11 Referências Bibliográficas 60

1 CAPITULO I: DESCRIÇÃO DO PROJECTO

1.1 Introdução

As instituições de ensino superior são espaços frequentados diariamente por estudantes, docentes e funcionários, que utilizam diferentes instalações, como salas de aula, bibliotecas, laboratórios, cantinas e áreas de convivência. Durante as actividades diárias, podem ocorrer situações em que objectos pessoais, como telemóveis, carteiras, documentos, chaves e materiais escolares, sejam perdidos ou encontrados por outras pessoas.

Quando não existe um mecanismo centralizado para registar e consultar esses objectos, a sua recuperação pode tornar-se difícil, uma vez que a informação pode ficar dispersa entre diferentes pessoas ou serviços da instituição.

Neste contexto, propõe-se o desenvolvimento de um Sistema de Gestão de Objectos Perdidos e Achados, destinado a facilitar o registo, a pesquisa e a recuperação de objectos pessoais dentro de uma instituição de ensino superior. O sistema permitirá aos utilizadores comunicar a perda de objectos, consultar os objectos encontrados e solicitar a sua recuperação. Por sua vez, o responsável pelos achados poderá registar os objectos encontrados, analisar os pedidos e confirmar a devolução dos mesmos.

1.2 Problematização

A gestão de objectos perdidos e encontrados numa instituição de ensino superior pode representar uma dificuldade quando não existem procedimentos organizados para o registo e a recuperação desses bens. Em determinadas situações, os objectos encontrados podem ser entregues informalmente a colegas, funcionários ou responsáveis por diferentes sectores, dificultando a sua localização posterior.

Esta situação pode provocar demora na recuperação dos objectos, perda de informações importantes e dificuldades na comunicação entre quem perdeu um objecto e quem o encontrou. Além disso, a ausência de um histórico organizado pode dificultar o acompanhamento dos pedidos e a confirmação das devoluções.

Perante este cenário, torna-se pertinente propor uma solução informática que centralize as informações e estabeleça procedimentos claros para o registo, a pesquisa, a solicitação de recuperação e a devolução dos objectos.

Deste modo, coloca-se a seguinte pergunta de partida:

Como pode um Sistema de Gestão de Objectos Perdidos e Achados melhorar o registo, a localização e a recuperação de objectos pessoais numa instituição de ensino superior?

1.3 Justificativa

O desenvolvimento deste sistema justifica-se pela necessidade de organizar o processo de gestão de objectos perdidos e encontrados, através de uma plataforma que permita centralizar as informações e facilitar a comunicação entre os intervenientes.

A solução proposta poderá reduzir as dificuldades na localização dos objectos, facilitar a pesquisa por categoria, descrição e local onde foram encontrados, bem como permitir o acompanhamento dos pedidos de recuperação. A confirmação da propriedade antes da devolução também contribuirá para tornar o processo mais organizado.

1.4 Objectivos

1.4.1 Objectivo geral

- Desenvolver um Sistema de Gestão de Objectos Perdidos e Achados para uma instituição de ensino

1.4.2 Objectivos específicos

- Identificar e especificar os requisitos funcionais, não funcionais e as regras de negócio do sistema.
- Modelar o funcionamento do sistema através de diagramas UML, incluindo casos de uso, actividade, classes e sequência.

- Definir a estrutura dos dadsos e os relacionamentos entre as principais entidades do sistema.
- Elaborar um protótipo das interfaces, de modo a representar as principais funcionalidades e a forma de interacção dos utilizadores com o sistema.
- Definir casos de teste que permitam verificar se as funcionalidades do sistema correspondem aos estabelecidos.

2 CAPITULO II: ESPECIFICAÇÃO DE REQUISITOS

2.1 Actores do Sistema

Os actores representam as pessoas que irão interagir directamente com o sotema para realizar determinadas operações. Para este projecto, foram identificados três actores principais

Actor 1 – utilizador

Representa um estudante, docente ou funcionário da instituição. Poderá registar objectos perdidos, consultar os objectos encontrados, pesquisar informações e solicitar a recuperação de um objecto.

Actor 2 – Responsável pelos achados

É o responsável pela gestão dos objectos encontrados. Poderá registar novos achados. Consultar os pedidos de recuperação, analisar as informações apresentadas pelos utilizadores e conformar a devolução dos objectos.

Actor 3 – Administrador

É responsável pela administração do sistema, incluindo a gestão das contas de utilizadores e dos respectivos acessos.

2.2 Funcionalidades Principais do Sistema

O sistema de Gestão de Objectos Perdidos e achados deverá disponibilizar as seguintes funcionalidades:

- Criar uma conta de utilizador e iniciar sessão.
- Registar objectos perdidos.
- Registar objectos encontrados.
- Consultar os objectos registados.

- Pesquisar objectos por descrição, categoria, local e estado.
- Solicitar a recuperação de um objecto encontrado.
- Analisar e aprovar ou rejeitar pedidos de recuperação.
- Actualizar o estado dos objectos.
- Confirmar a devolução dos objectos aos respectivos proprietários.
- Consultar o histórico de objectos e pedidos, de acordo com as permissões.
- Gerir as contas de utilizadores.

2.3 Requisitos funcionais

Os requisitos funcionais descrevem as oprações que o sistema deverá executar para responder às necessidadess dos seus utilizadores.

RF01 – Registo de utilizador

O sistema deve permitir que uma pessoa crie uma conta, fornecendo os dados necessários para a sua identificação e acesso.

RF02 – Autenticação

O sistema deve permitir que os utilizadores iniciem e terminem sessão através das suas credenciais de acesso.

RF03 – Registo de Objectos Perdidos

Os sistema deve permitir que um utilizador registe um objecto perdido, indicando a descrição, a categoria, a data e o local provável da perda.

RF04 – Registo de Objectos Encontrados

O sistema deve permitir que o responsável pelos achados registe um objecto encontrado, indicando a descrição, a categoria, a data e o local onde foi encontrado.

RF05 – Consulta de Objectos

O sistema deve permitir que os utilizadores consultem os objectos perdidos e encontrados registados na plataforma, respeitando as permissões de acesso.

RF06 – Pesquisa e Filtragem

O sistema deve permitir pesquisar objectos através de critérios como descrição. Categoria, local e estado.

RF07 – Solicitação de Recuperação

O sistema deve permitir que um utilizador solicite a recuperação de um objecto encontrado, fornecendo informações que ajudem a comprovar a sua propriedade.

RF08 – Análise dos Pedidos

O sistema deve permitir que o responsável pelos achados consulte os pedidos de recuperação e registe a sua decisão, após analisar as informações apresentadas.

RF09 – Actualização do Estado dos Objectos

O sistema deve permitir que o responsável pelos achados actualize o estado de um objecto, de acordo com a fase do processo de recuperação.

RF10 – Confirmação da Devolução

O sistema deve permitor que o responsável pelos achados e perdidos registe a devolução de um objecto depois de confirmar a identidade ou a propriedade do bem pelo requerente.

RF11 – Consulta do Histórico

O sistema deve permitir a consulta do histórico dos objectos e dos pedidos de recuperação, de acordo com as permissões atibuídas a cada actor.

RF12 – Gestão de utilizadores

O sistema deve permitir que o administrador consulte, active ou desactive contas de utilizadores.

2.4 Requisitos Não Funcionais

Os requisitos não funcionais definem as condições de qualidade, segurança e desempenho que o sistema deverá respeitar.

RNF01 – Segurança

O sistema deve restringir o acesso às funcionalidade administrativas e às operações reservadas aos utilizadores autenticados, de acordo com as respectivas permissões.

RNF02 – Usabilidade

A interface deve apresentar menus e formulários simples, com mensagens de cnfirmação e de erro compreensíveis .

RNF03 – Desempenho

Em condições normais de utilização e com volume de dados definido para os testes, as pesquisas de objectos deverão apresentar resultados em até três segundos. Este valor constitui uma meta a verificar, não um resultado ja comprovado.

RNF04 – Integridade de Dados

O sistema deve validar os dados introduzidos e impedir registos inconsistentesm recorrendo a regras de validação e restrições na base de dados.

RNF05 – Manutenibilidade

O sistema deve apresentar uma organização do código que facilite a identificação, a correção de erros e a introdução de novas funcionalidades.

2.5 Regras de negócio

As regras de megócio estabelecem as condições que devem ser respeitadas durante a utilização do sistema.

RN01 – Autenticação Obrigatória

As operações que exigem identificação do utilizador só poderão ser realizadas após uma autenticação válida.

RN02 – identificação dos objectos

Cada objecto registado deve possuir um identificador único e uma descrição que permita distingui-lo dos restantes registos.

RN03 – Verficação da propriedade

Um pedido de recuperação só poderá ser aprovado depois do responsável pelos achados verificar se as informações apresentadas são suficientes para sustentar a propriedade do objecto.

RN04 – Autorização da devolução

A confirmação da devolução de um objecto será uma operação exclusiva do responsável pelos achados.

RN05 – Objectos já devolvidos

Um objecto cujo estado seja «Devolvido» não poderá receber novos pedidos de recuperação enquanto permanecer nesse estado.

RN06 – Coerência do estado

Os estado de um objecto deve reflectir a sua situação no processo, não podendo ser considerado disponível para recuperação depois de sua devolução ter sido confirmada.

RN07 – Registo de pedidos

Cada pedido de recuperação deve estar associado ao utilizador que o submeteu e ao objecto a que a se refere, permitindo acompanhar a decisão tomada.

3 CAPITULO III: ARQUITECTURA DO SISTEMA

3.1 Modelo de Desenvolvimento

A engenharia de software requer uma abordagem metódica, disciplinada e mensurável na criação, operação e manutenção de programas. A demanda crescente por sistemas eficientes e ágeis no desenvolvimento de projectos aumenta a adoração das metodologias ágeis, como o Extreme Programming (XP) e o Scrum (Pablos & Feitosa, 2020). Neste sentido, a engenharia de software ágil tem como foco a satisfação do cliente e a entrega progressiva do projecto, priorizando as necessidades mais urgentes do cliente (Pressman, 2021).

O desenvolvimento incremental é um exemplo de metodologia ágil, que divide o projecto em partes completas, construídas e entregues de forma sequencial. Cada etapa é avaliada pelo cliente e, caso atenda aos requisitos, avança para a próxima. Essa abordagem pode ocasionar atrasos e perda do foco principal do projecto, mas pode ser aperfeiçoada para minimizar os danos e aumentar a eficiência (Pressman, 2021). Contudo, a seleção do modelo de processo de desenvolvimento de software adequado não é trivial, pois cada modelo possui limitações e nem sempre se aplica a todas as situações (Wazlawick, 2013). Alem disso, a escolha do modelo pode ser uma das causas de fracasso nos projectos de software (Sommerville, 2021). Para selecionar o modelo apropriado, é preciso entender as necessidades do projecto e analisar os diferentes modelos existentes (Peters & Pedrycz, 2001).

O desenvolvimento incremental é considerado uma evolução do modelo em cascata, trazendo vantagens que seu antecessor não oferece (Dias, 2019). Dentre os benefícios do modelo incremental em comparação ao modelo em cascata, estão a redução do custo das mudanças, maior facilidade para obter feedback dos clientes, entregas mais rápidas, envolvimento do usuário e

melhor gerenciamento dos riscos do desenvolvimento. Entretanto, a estrutura do sistema pode se deteriorar com a adição de novas etapas e problemas complexos podem ser postergados (Bezerra, 2014; Sommerville et al., 2021)

Para o desenvolvimento do sistema de Gestão de Objectos Perdidos e Achados, propõe-se a utilização do modelo de desenvolvimento incremental. Este modelo permite construir o sistema por etapas, acrescentando e verificando funcionalidades de forma progressiva.

A escolha justifica-se pela necessidade de organizar o desenvolvimento em partes menores, facilitando a identificação de erros e a realização de testes durante o processo. Por outro lado, permite dar prioridade às funcionalidades essenciais, como o registo de objectos, a pesquisa, a solicitação de recuperação e a confirmação da devolução.

O desenvolvimento será organizado nos seguintes incrementos:

- **Primeiro incremento:** criação da base de dados e implementação da autenticação e gestão de utilizadores.
- **Segundo incremento:** implementação do registo, consulta e pesquisa de objectos perdidos e encontrados.
- **Terceiro incremento:** implementação dos pedidos de recuperação, análise dos pedidos e confirmação da devolução.
- **Quarto incremento:** realização de testes, correcção de erros e melhoria da interface.

Desta forma, cada incremento acrescentará funcionalidades ao sistema até se obter uma versão funcional das operações principais.

3.2 Arquitectura do Sistema

Para este projecto, propõe-se uma arquitectura em camadas, que organiza o sistema em componentes com responsabilidades distintas. Esta organização facilita a comprreensão do código, a auntenticação e a identificação de erros.

A arquitectura será composta por quatro camadas principais:

a) Camada de apresentação

É responsável pela interface gráfica utilizada pelos actores. Disponibilizará formulários, menus, campos de pesquisa e mensagens para o registo e consulta de informações.

b) Camada de lógica de negócio

Contém as regras que determinam o funcionamento do sistema, incluindo a validação dos dados, a análise dos dos pedidos de recuperação e as condições necessárias para confirmar a devolução de um objecto.

c) Camada de acesso a dados

É responsável pela comunicação entre a aplicação e a base de dados. Executará operações de inserção, consulta, actuaalização e, quando permitido, remoção de registos.

d) Camada de dados

Corresponde à base dados, onde serão armazenadas as informações dos utilizadores, dos objectos e dos pedidos de recuperação. A separação destas camadas permite que alterações na interface sejam realizadas sem exigir, necessariamente, alterações na base de dados ou em todas as regras de negócio.

3.3 Atributos de Qualidade

Os atributos de qualidade representam caracteristicas que contribuem para o funcionamento adequado do sistema. Para este projecto, foram seleccionados cinco atributos principais

3.3.1 Segurança

O sistema deverá limitar o acesso às operações de acordo com o perfil do utilizador. As funcionalidades administrativas e a confirmação da devolução serão reservadas aos actores autorizados.

3.3.2 Usabilidade

A interface deverá ser simples e organizada, permitindo que os utilizadores realizem operações através de formulários e menus compreensíveis. As mensagens apresentadas deverão ajudar a identificar erros e confirmar operações.

3.3.3 Desempenho

O sistema deverá permitir consultar e pesquisar objectos sem demoras excessivas. Como meta de desempenho, pretende-se apresentar os resultados das pesquisas em até três segundos, nas condições de teste definidas.

3.3.4 Integração de dados

A aplicação deverá validar as informações introduzidas e manter a consistência dos registos. A base de dados deverá utilizar chaves primárias, chaves estrangeiras e restrições adequadas para evitar associações inválidas.

3.3.5 Manutenibilidade

O código deverá estar organizado em classes e camadas, separando a interface gráfica, as regras de negócio e o aceddo a dados. Esta organização facilitará a correção de erros e a introdução de novas funcionalidades.

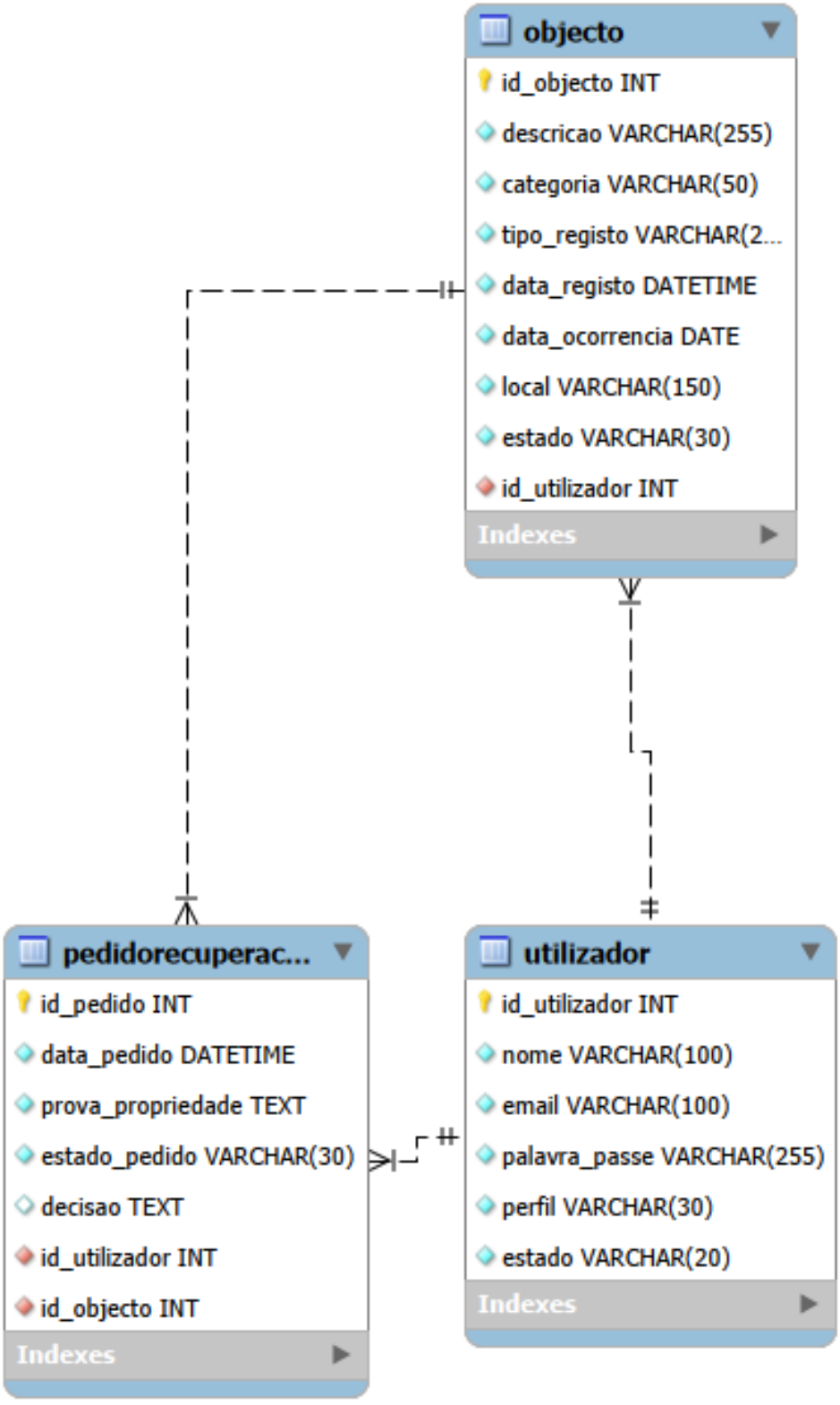
3.4 Linguagens e Tecnologias

Para implementação da solução, propôem-se as seguintes tecnologias e ferramentas:

Tecnologia ou Ferramenta	Finalidade
Java	Lkinguagem de programação utilizada no desenvolvimento da aplicação
Java Swing	Criação da interface gráfica da aplicação
Microsoft SQL Server Express	Armazenamento e gestão dos dados do sistema

JDBC	Comunicação entre a aplicação java e a base de dados.
Eclispse IDE	Escrita, organização e execução do código java.
Digrams.net (draw.io) e Planttext	Criação dos diagramas UML e do mdelo entidade-relacionamento.
Git e GitHub	Controlo de versões e registo das alterações realizadas no projecto.

3.5 Modelo Entidade – Relacionamento (MER)


objecto
id_objecto INT
descricao VARCHAR(255)
categoria VARCHAR(50)
tipo_registo VARCHAR(2...
data_registo DATETIME
data_ocorrencia DATE
local VARCHAR(150)
estado VARCHAR(30)
id_utilizador INT
Indexes
pedidorecuperac...
id_pedido INT
data_pedido DATETIME
prova_propriedade TEXT
estado_pedido VARCHAR(30)
decisao TEXT
id_utilizador INT
id_objecto INT
Indexes
utilizador
id_utilizador INT
nome VARCHAR(100)
email VARCHAR(100)
palavra_passe VARCHAR(255)
perfil VARCHAR(30)
estado VARCHAR(20)
Indexes

3.6 Identificação das entidades

Para o funcionamento do sistema, identificámos três entidades principais: utilizador, objecto e PedidoRecuperacao.

Entidade	Descrição
Utilizador	Armazena os dados dos utilizadores do sistema, cinluindo estudantes, docentes, funcionários e responsáveis pelos achados
Objecto	Armazena as informações sobre os objectos perdidos e encontrados na instituição.
PedidoRecuperacao	Regista os pedidos efectuados pelos utilizadores para recuperar objectos encontrados.

3.6.1 Atributos das entidades

Cada entidade possui atributos que permitem armazenar e identificar as respectivas informações.

Entidade Utilizador

Atributo	Descrição
Id_utilizador (PK)	Identificador único do utilizador.
nome	Nome completo do utilizador
email	Endereço electrónico do utilizador
palavra_passe	Palavra-passe utilizada para auntenticação
perfil	Papel do utilizador no sistema
estado	Indica se a conta está activa ou inactiva.

Entidade Objecto

Atributo	Descrição
id_objecto (PK)	Identificador único do projecto
descricao	Descrição do objecto perdido ou encontrado
categoria	Categoria a que pertence o objecto
tipo_registo	Indica se o objecto foi registado como perdido ou encontrado
data_registo	Data em que o objecto foi registado no sistema
data_ocorrencia	Data em que o objecto foi perdido ou encontrado
local	Situação actual do objecto
estado	Situação actual do objecto
id_utilizador (FK)	Referência ao utilizador que registou o objecto.

Entidade PedidoRecuperacao

Atributo	Descrição
Id_pedido (PK)	Indentificador único do pedido
data_pedido	Data em que o pedido foi efectuado
prova_propriedade	Informação ou descrição da prova apresentada para demostrar a propriedade do objecto.
estado_pedido	Situação actual do pedido
decisao	Decisão tomada pelo responsável relativamente ao pedido
id_utilizador (Fk)	Referência ao utilizador que efectuou o pedido.

id_objecto (FK)	Referência ao objecto que se pretende recuperar

PK (primary key): chave primária, que identifica unicamente cada registo.

FK (Foreign Key): chave estrangeira, que estabelece uma ligação entre tabelas.

3.7 Relacionamentos e cardinalidades

As entidades relacionam-se da seguinte maneira:

3.7.1 Utilizador – Objecto

Um utilizador pode registar vários objectos, mas cada objecto registado está associado a um utilizador.

Cardinalidade: 1:N

3.7.2 Utilizador – PedidoRecuperacao

Um utilizador pode efectuar vários pedidos de recuperação, mas cada pedido pertence a um único utilizador.

Cardinalidade: 1:N

3.7.3 Objecto – PedidoRecuperacao

Um objecto pode estar associado a vários pedidos de recuperação, mas cada pedido refere-se a um único objecto.

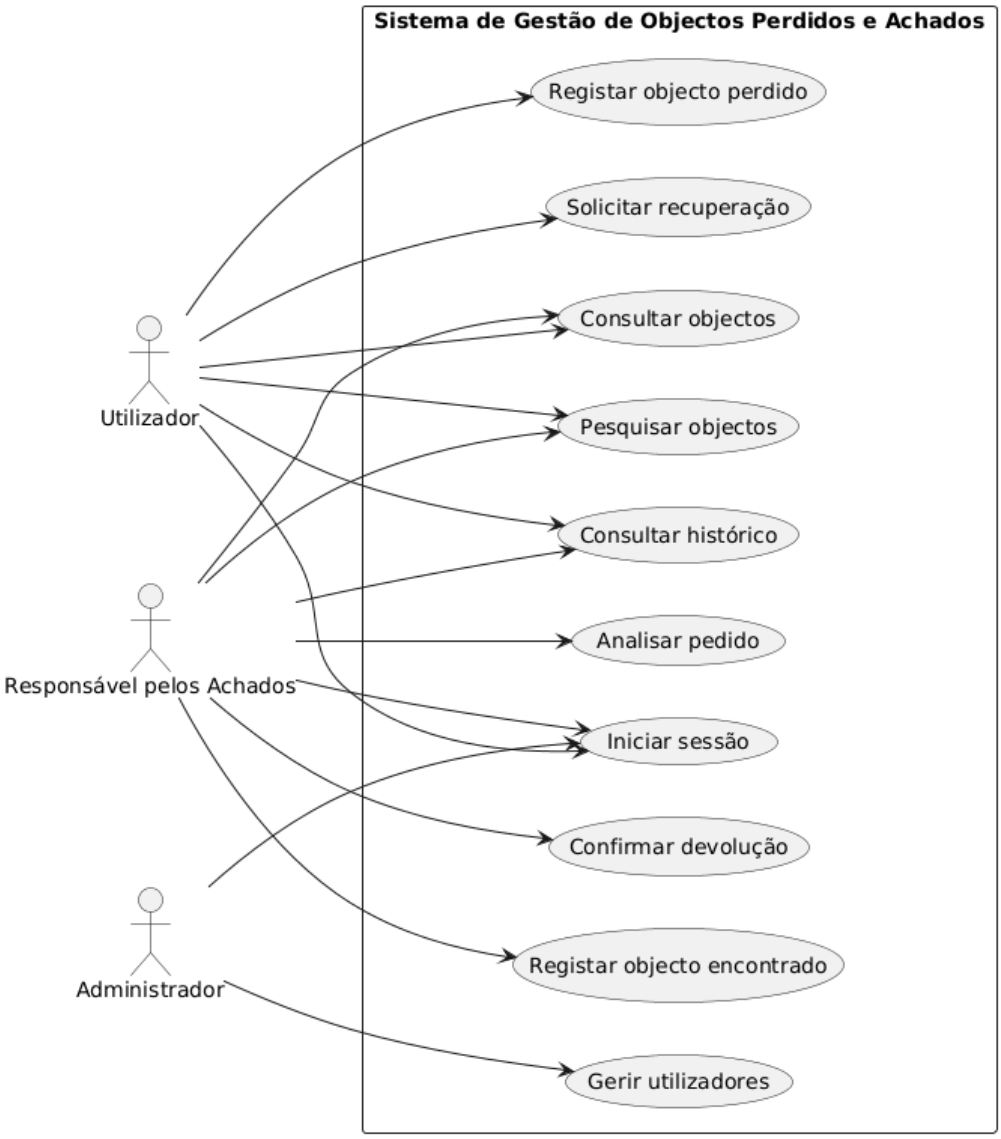
Cardinalidade: 1:N

4 CAPITULO IV: DESENHO E IMPLEMENTAÇÃO

4.1 Diagrama de caso de uso

Diagrama de casos de uso permite representar as funcionalidades de um sistema na perspectiva do utilizador, identificando as principais operações que o sistema disponibiliza e a forma como os utilizadores interagem com essas funcionalidades (WordPress, 2019).

Segundo sommerville (2018), os diagramas de casos de uso apresentam uma visão geral e simplicicadadas interações com o sistema ,sendo necessário complemenytar essa representação com informações adicionais para descrever as funcionalidades de forma mais completa.


Sistema de Gestão de Objectos Perdidos e Achados
Registar objecto perdido
Solicitar recuperação
Consultar objectos
Pesquisar objectos
Consultar histórico
Analisar pedido
Iniciar sessão
Confirmar devolução
Registar objecto encontrado
Gerir utilizadores
Utilizador
Responsável pelos Achados
Administrador

Descrição do diagrama

O diagrama apresenta as funcionalidades disponibilizadas a cada actor.

O Utilizador interage com o sistema para registar objectos perdidos, consultar os registos existentes e solicitar a recuperação de objectos encontrados. O Responsável pelos Achados gere os objectos encontrados e acompanha os pedidos de recuperação até à confirmação da devolução. O Administrador gere as contas e os acessos dos utilizadores.

A separação das responsabilidades permite organizar o funcionamento do sistema e restringir determinadas operações aos actores autorizados.

4.1.1 Especificação dos casos de uso

Caso de uso 01 – Registar Objecto Perdido

Actor principal: Utilizador

Objectivo: Permitir que o utilizador registe um objecto que perdeu nas instalações da instituição.

Pré-condições

- O utilizador deve ter uma conta activa
- O utilizador deve ter iniciado sessão

Fluxo principal:

1. O utilizador acede à opção de registo de objecto perdido
2. O sistema apresenta o formulário de registo.
3. O utilizador introduz a descrição. A categoria, a data e o local provável da perda.
4. O utilizador submete o formulario
5. O sistema valida os dados introduzidos
6. O sistema guarda o registo na base de dados.
7. O sistema apresenta uma mensagem de confirmação

Fluxo alternativo

Se existirem campos obrigatórios por preencher ou dados inválidos, o sistema apresenta uma mensagem de erro e solicita a correção das informações.

Pós-condições

- O objecto fica registado na base de dados.
- O registo fica associado ao utilizador que o criou.

Caso de Uso 02 — Registar Objecto Encontrado

Actor principal: Responsável pelos Achados.

Objectivo: Registar um objecto encontrado nas instalações da instituição para facilitar a sua identificação e recuperação.

Pré-condições:

- O responsável deve possuir uma conta activa.
- O responsável deve ter iniciado sessão.

Fluxo principal:

1. O responsável acede à opção de registo de objecto encontrado.
2. O sistema apresenta o formulário correspondente.
3. O responsável introduz a descrição, a categoria, a data e o local onde o objecto foi encontrado.
4. O responsável submete os dados.
5. O sistema valida as informações.
6. O sistema guarda o registo na base de dados.
7. O sistema apresenta uma mensagem de confirmação.

Fluxo alternativo:

Se os dados forem inválidos ou estiverem incompletos, o sistema informa o responsável e solicita a sua correcção.

Pós-condições:

- O objecto encontrado fica registado.
- O objecto fica disponível para consulta, de acordo com as regras de acesso definidas.

Caso de Uso 03 — Solicitar Recuperação de Objecto

Actor principal: Utilizador.

Objectivo: Permitir que um utilizador solicite a recuperação de um objecto que considera ser da sua propriedade.

Pré-condições:

- O utilizador deve ter iniciado sessão.
- O objecto deve estar registado no sistema.
- O objecto não pode estar marcado como devolvido.

Fluxo principal:

1. O utilizador pesquisa os objectos encontrados.
2. O sistema apresenta os resultados correspondentes.
3. O utilizador selecciona o objecto pretendido.
4. O sistema apresenta os detalhes disponíveis.
5. O utilizador selecciona a opção de solicitar recuperação.
6. O utilizador fornece informações que ajudem a comprovar a propriedade do objecto.
7. O utilizador submete o pedido.
8. O sistema regista o pedido e associa-o ao utilizador e ao objecto.

9. O sistema apresenta uma mensagem de confirmação.

Fluxo alternativo:

Se o objecto já tiver sido devolvido ou os dados obrigatórios estiverem incompletos, o sistema impede a submissão do pedido e apresenta uma mensagem explicativa.

Pós-condições:

- O pedido fica registado com o estado «Pendente».
- O pedido fica disponível para análise pelo Responsável pelos Achados.

Caso de Uso 04 — Analisar Pedido de Recuperação

Actor principal: Responsável pelos Achados.

Objectivo: Verificar as informações apresentadas pelo requerente e decidir se o pedido de recuperação pode ser aprovado.

Pré-condições:

- O responsável deve ter iniciado sessão.
- Deve existir um pedido de recuperação pendente.

Fluxo principal:

1. O responsável acede à lista de pedidos pendentes.
2. O sistema apresenta os pedidos registados.
3. O responsável selecciona um pedido.
4. O sistema apresenta os dados do pedido e do objecto associado.
5. O responsável analisa as informações fornecidas.
6. O responsável regista a decisão de aprovar ou rejeitar o pedido.
7. O sistema guarda a decisão.
8. O sistema actualiza o estado do pedido.

Fluxo alternativo:

Se as informações não forem suficientes para confirmar a propriedade, o responsável rejeita o pedido ou solicita informações adicionais, caso essa funcionalidade seja implementada.

Pós-condições:

- A decisão fica registada.
- O estado do pedido é actualizado para «Aprovado» ou «Rejeitado».

Caso de Uso 05 — Confirmar Devolução

Actor principal: Responsável pelos Achados.

Objectivo: Registar a devolução de um objecto ao seu legítimo proprietário.

Pré-condições:

- O responsável deve ter iniciado sessão.
- O pedido de recuperação deve ter sido aprovado.
- A propriedade do objecto deve ter sido verificada.

Fluxo principal:

1. O responsável consulta o pedido aprovado.
2. O responsável confirma a entrega do objecto ao requerente.
3. O responsável selecciona a opção de confirmar devolução.
4. O sistema verifica se a operação está autorizada.
5. O sistema actualiza o estado do objecto para «Devolvido».
6. O sistema regista a confirmação da devolução.
7. O sistema apresenta uma mensagem de sucesso.

Fluxo alternativo:

Se o pedido não estiver aprovado ou a operação for realizada por um actor sem autorização, o sistema impede a confirmação da devolução.

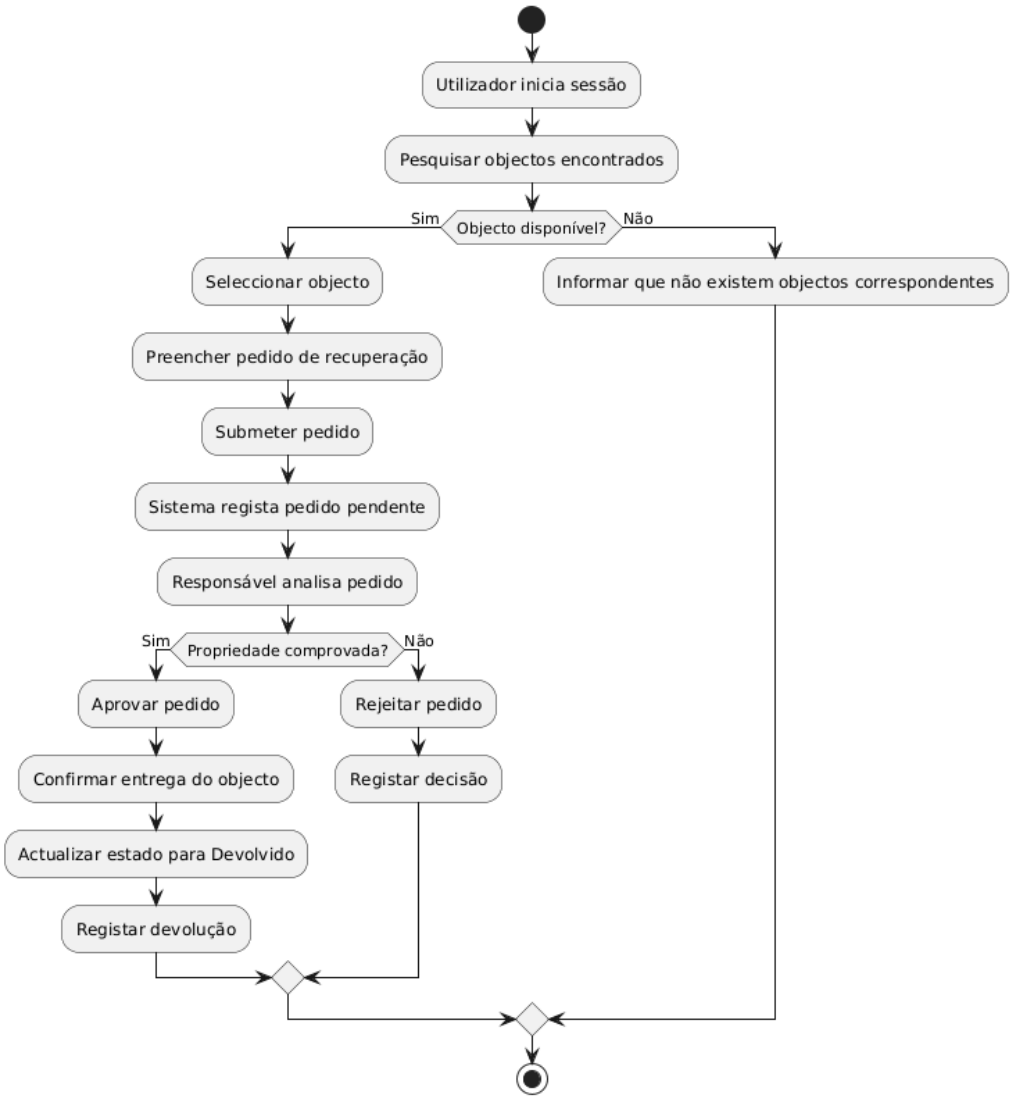
Pós-condições:

- O objecto fica marcado como «Devolvido».
- A devolução fica registada no sistema.
- O objecto deixa de aceitar novos pedidos de recuperação.

4.2 Diagrama de Actividades

De acordo com WordPress (2019), o diagrama de actividades tem como principal objectivo especificar o comportamento funciomal do software, representando as funcionalidades e os processos envolvidos no seu funcionamento. Estediagrama permite documentar os aspectos funcionais do software, ilustrar etapas específicas de processo de negócio automatizados, demonstrar como como as funcionalidades contribuem para a realização dos requisitos funcionais e descrever, de forma geral, o funcionamento do sistema.

O diagrama de actividades representa a sequência de acções realizadas durante um processo. Neste projecto, será utilizado para ilustrar o processo de recuperação de um objecto encontrado. O processo começa com a pesquisa de objectos pelo utilizador e termina quando o pedido é rejeitado ou quando o objecto é devolvido após a aprovação do pedido.



Descrição do Processo

O processo inicia-se quando o utilizador acede ao sistema e pesquisa os objectos encontrados. Se encontrar um objecto correspondente, poderá submeter um pedido de recuperação, fornecendo informações que ajudem a comprovar a propriedade. O pedido fica pendente até ser analisado pelo responsável pelos Achados. Se as informações forem suficientes, o pedido poderá ser aprovado e a devolução será confirmada. Caso contrário, o pedido será rejeitado.

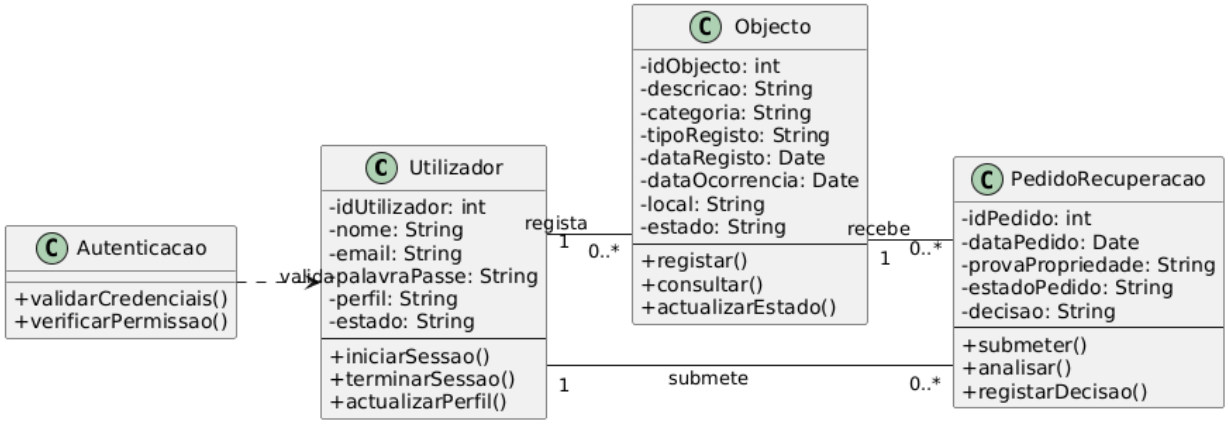
4.3 Diagrama de Classes

Os diagramas de classe são utilizados na modelação de sistemas orientados a objectos, permitindo representar as classes que constituem o sistema e as relações existentes entre elas. Uma classe define as caracteristicas gerais de um tipo de objecto, enquanto uma associação representa a ligação ou o relacionamento entre duas ou mais classes. (Sommerville, 2018).

De acordo com WordPress (2019), os diagramas de classes também apresentam vanatgens para as organizações, pois facilitam a compreensão da estrutura de uma aplicação, permitindo visualizar melhor os requisitos específicos do sistema e ajudam a partilhar informações sobre a sua organização e funcionamento.

O diagrama de classes representa a estrutura do sistema através das classes, dos seus atributos, dos métodos e dos relacionamentos.

Para a primeira versão, são consideradas as classes Utilizador, Objecto, PedidoRecuperação e Autenticação. A classe Utilizador representa os actores com conta no sistema, enquanto as calsses Objecto e PedidoRecuperação representam os dados centrais do processo. A classe Autenticação será responsável por apoiar a validação das credeciais de acesso.



Descrição das classes

a) Classe Utilizador

Armazena os dados de identificação e acesso dos utilizadores do sistema. O atributo perfil permite distinguir os diferentes níveis de acesso, como utilizador comum, responsável pelos achados e administrador. Os seus métodos permitem apoiar a autenticação, o encerramento da sessão e a actualização dos dados pessoais.

b) Classe Objecto

Representa os objectos perdidos ou encontrados que são registados no sistema. Contém informações como descrição, categoria, tipo de registo, datas, local e estado. Os seus métodos permitem registar, consultar e actualizar as informações dos objectos, respeitando as permissões e as regras de negócio.

c) Classe PedidoRecuperacao

Representa os pedidos submetidos pelos utilizadores para recuperar objectos encontrados. Armazena a data do pedido, as informações de prova de propriedade, o estado e a decisão tomada. Os seus métodos permitem submeter pedidos, analisá-los e registar as decisões do responsável.

d) Classe Autenticacao

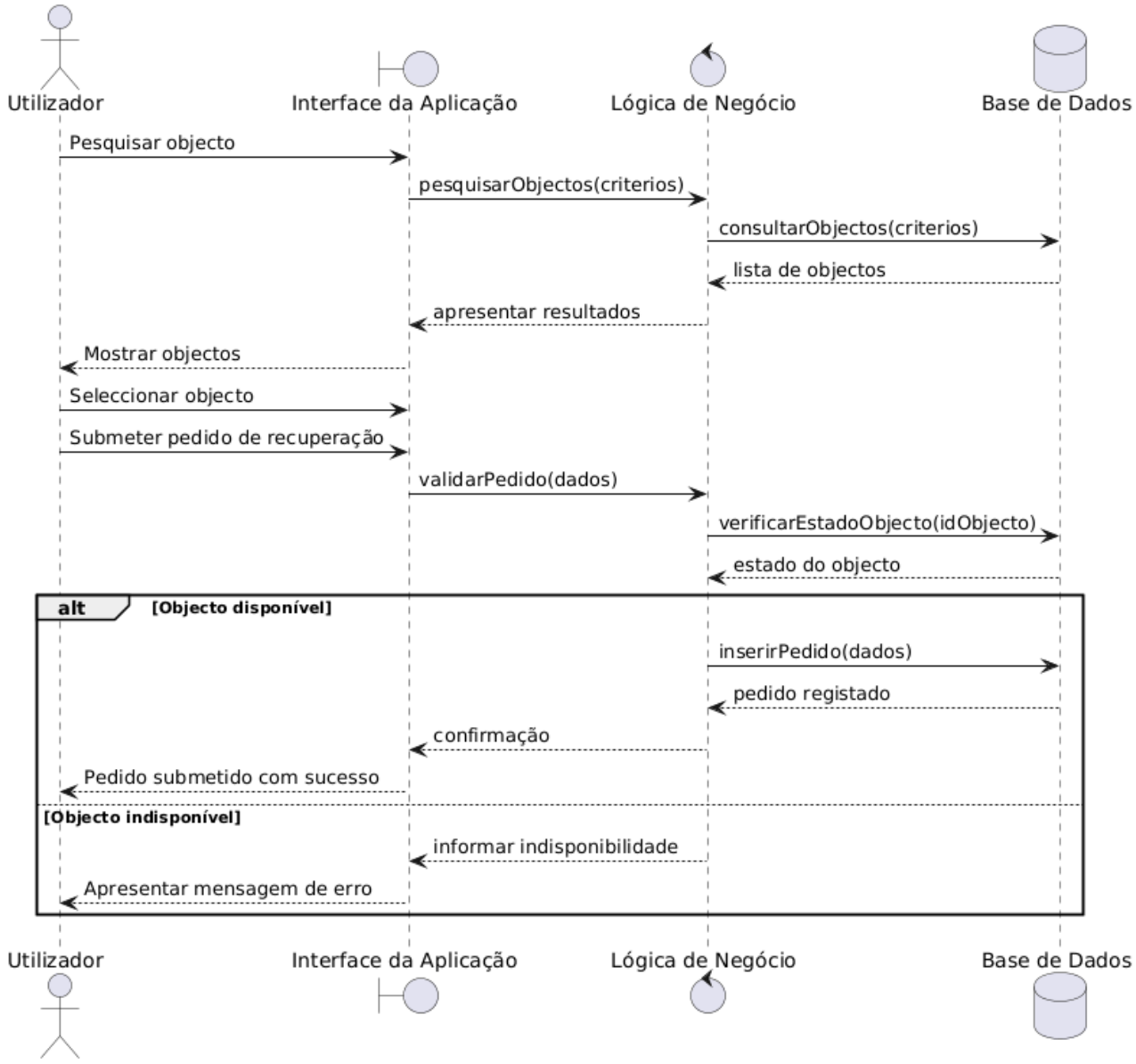
Responsabiliza-se pelas operações relacionadas com a validade das credenciais e a verificação das permissões de acesso. A sua separação permite organizar as operações de segurança e evitar a repetição da mesma lógica em deiferentes partes da aplicação.

4.4 Diagrama de Sequência

Segundo WordPress (2019), o diagram de sequência da UML representa a ordem em que os processos ocorrem num sistema, mostrando como os diferentes objectos colaboram para executar determinadas acções ao longo do tempo. Para sommerville (2018), este diagrama é

utilizado para modelar as interações entre os actores e os objectos de um sistema, bem como as interecções entre os actores e os objectos de um sistema, bem como as interacções estabelecidas entre os próprios objectos.

O diagrama de sequência representa a troca de mesnagens entre os participantes de um processo, respeitando a ordem temporal das operações. Neste projecto, será apresentado o processo de solicitação de recuperação de um objecto. O diagrama mostra a interacção entre o utilizador, a interface da aplicação, a lógica de negócio e a base de dados.



Descrição da sequência

O processo começa quando o utilizador pesquisa um objecto encontrado. A interface envia os critérios de pesquisa à lógica de negócio, que consulta a base de dados e devolve os resultados encontrados. Depois de seleccionar um objecto, o utilizador submete um pedido de recuperação, fornecendo informações que ajudem a comprovar a propriedade. A lógica de negócio verifica se o objecto está disponível e, caso estejam reunidas as condições necessárias, regista o pedido na base

de dados. Se o objecto não estiver disponível para recuperação, o sistema apresenta uma mensagem a informar que não é possível prosseguir com a operação.

4.5 Protótipo das Interfaces

Segundo sommerville (2018), um protótipo é uma versão inicial de um sistema, desenvolvida com o objectivo de demostrar conceitos, testar alternativas de projecto e aprofundar a compreensão do problema e das possíveis soluções. De igual modo, os protótipos permitem que os potenciais utilizadores avaliem em que medida o sistema pode responder às necessidades e apoiar a realização das suas tarefas.

Por sua vez, Pressman e Maxin (2021), dizem que apesar da rototipagem facilitar a compreensão do funcionamento do software por parte dos utilizadores, apresenta algumas desvantagens. Uma delas é a possibilidade dos envolvidos confundirem o protótipo com a verssão final do sistema, esquecendo-se de que a sua arquitectura ainda está em desenvolvimento e de que a qualidade global do software ainda foi devidamente assegurada. Outra desvantagem é a possibilidade do engenheiro de software adoptar soluções simplificadas ou comprometer determinados aspectos da implementação para colocar o protótipo em funcionamento num curto espaço de tempo.

O protótipo das interfaces do Sistema de Gestão de Objectos Perdidos e Achados foi concebido com o objectivo de representar visualmente as principais funcionalidades definidas nos requisitos do sistema. Para a sua elaboração, foi utilizado o figma, uma ferramenta que permite gerar interfaces.

O protótipo contempla as seguintes interfaces:

a) **Interface de autenticação** e registo de utilizadores: permite representar a criação de contas, o início de sessão e o acesso ao sistema de acordo com o perfil do utilizador.

b) **Página principal:** apresenta um resumo da situação dos objectos registados, dos objectos encontrados, dos objectos devolvidos e dos pedidos de recuperação pendentes
c) **Interface de registo de objectos**: permite representar o registo de objectos perdidos e encontrado, através de formulários com informações como descrição, categoria, data, local e estado.
d) **Interface de consulta e pesquisa**: permite visualizar os objectos registados e pesquisar através de critérios como descrição, categoria, local e estado.
e) **Interface de detalhes e recuperação de objectos**: apresenta as informações de cada objecto e permite representar a submissão de pedidos de recuperação, incluindo a justificação e as informações necessárias para comprovar a propriedade.
f) **Interface de gestão dos pedidos**: permite visualizar os pedidos de recuperação e representar a análise, aprovação ou rejeição dos mesmos.
g) **Interface de gestão de estados e devoluções**: representa a actualização do estado dos objectos e a confirmação da sua devolução aos respectivos proprietários.
h) **Interface de histórico**: permite representar a consulta do histórico dos objectos e dos pedidos de recuperação, respeitando as permissões dos utilizadores.
i) **Interface de administração**: representa a consulta, activação e desactivação das contas dos utilizadores.

Figura 1: interface de autenticação do sistema

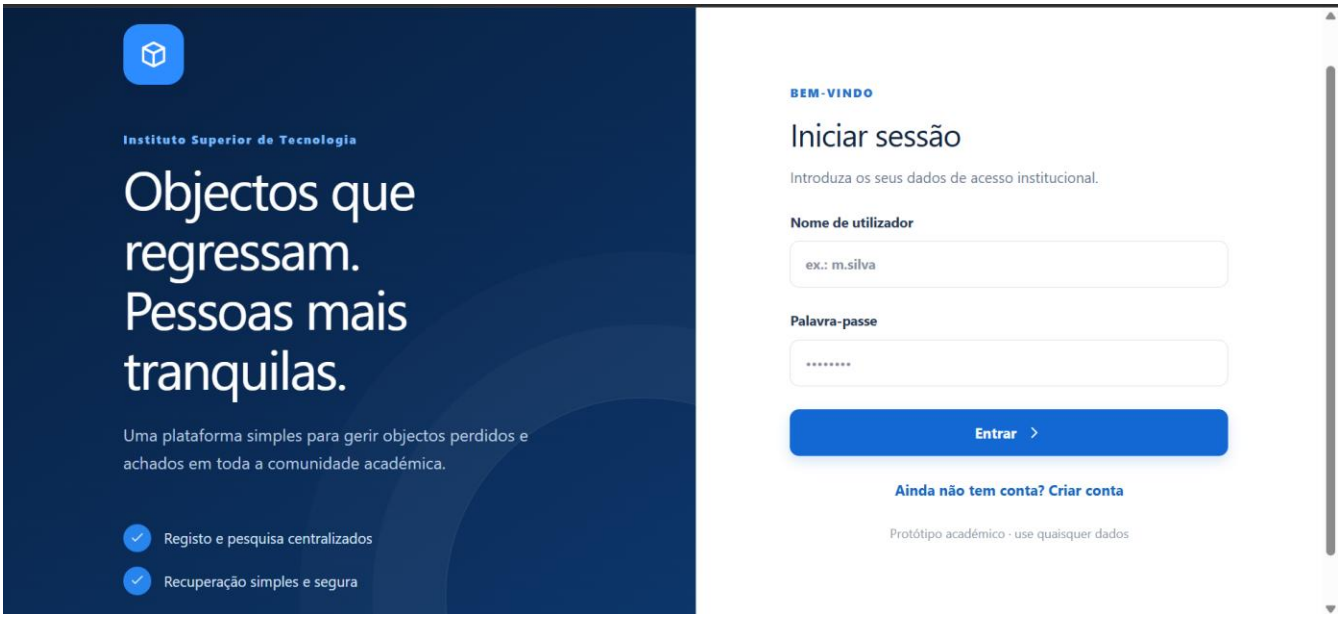


Figura 2: página principal do sistema

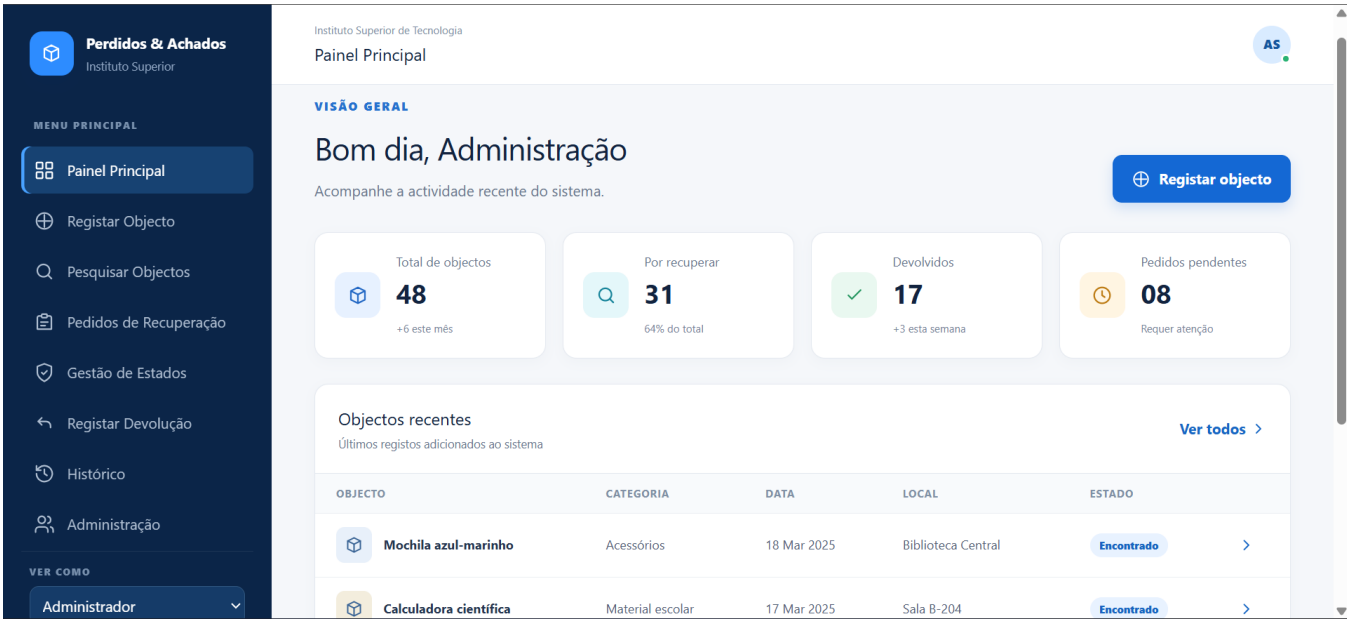


Figura 3: interface de registo de objectos



Figura 4: interface de consulta e pesquisa de objectos.

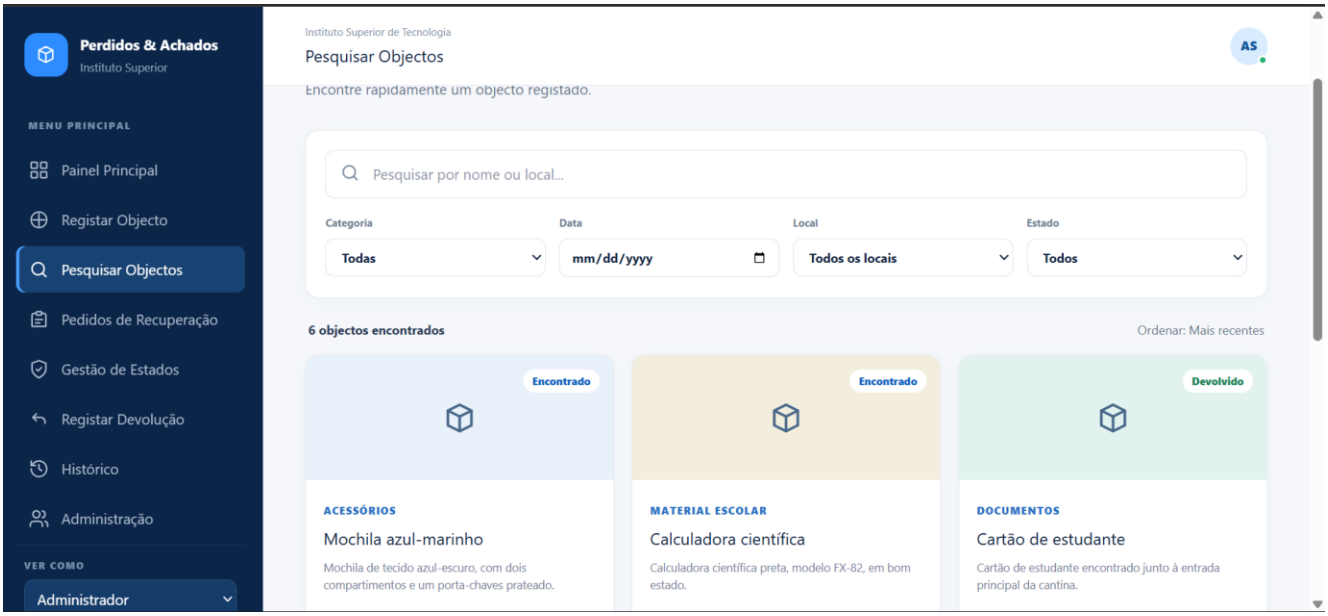


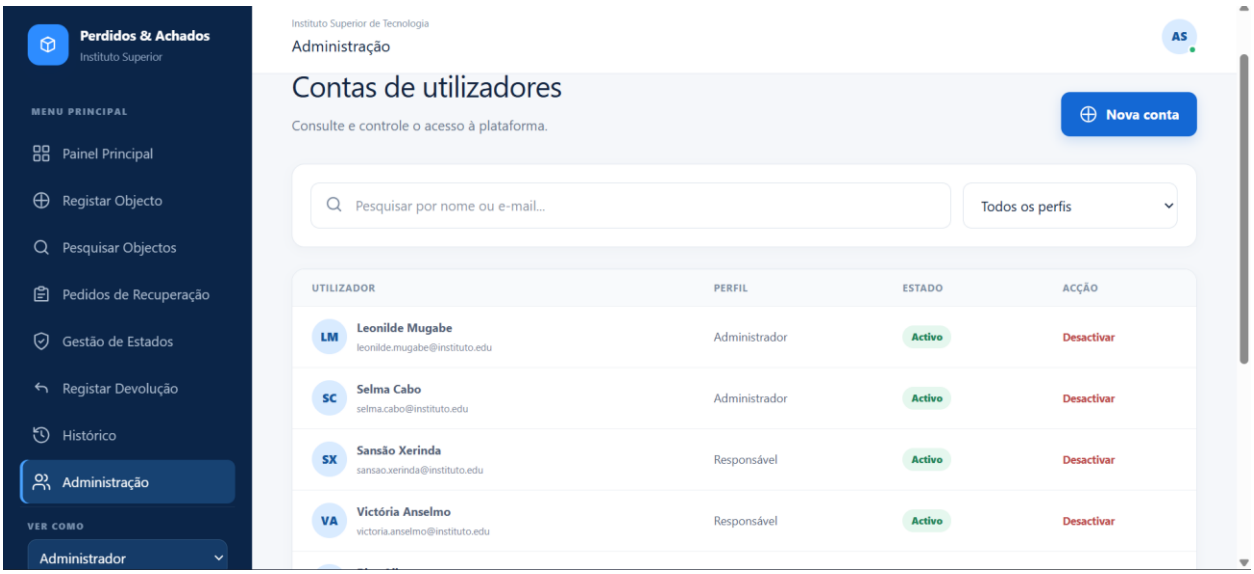
Figura 5: interface de gestão dos pedidos de recuperação.



Figura 6: interface de gestão de estados e devoluções



Figura 7: interface de administração e histórico



Interface do Utilizador

Figura 8: painel principal do utilizador

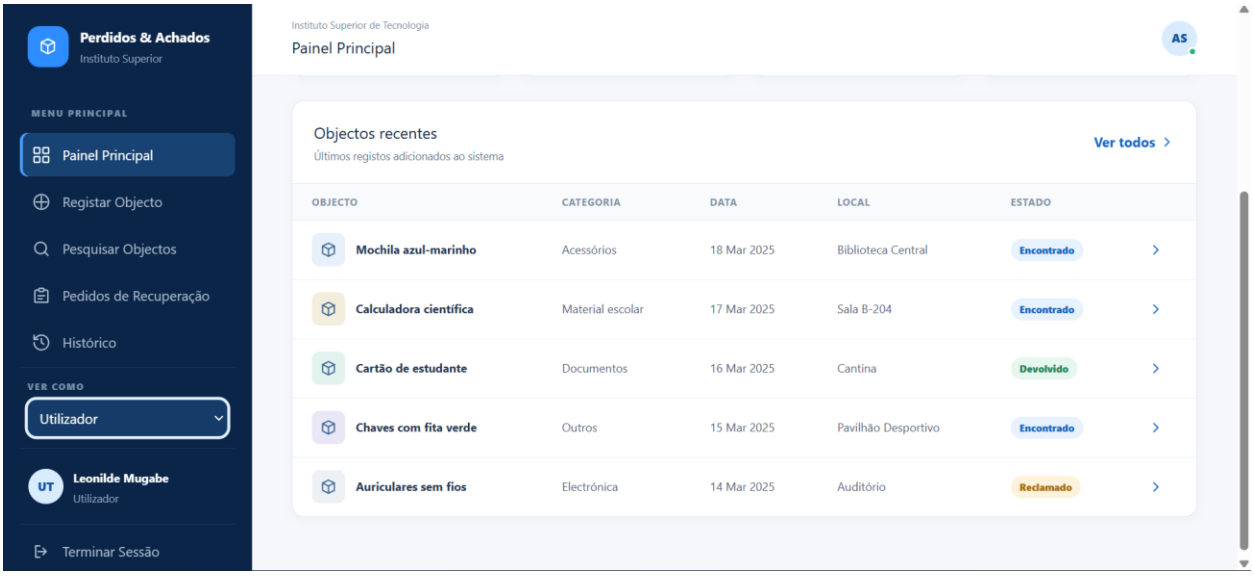


Figura 9: Registo de objecto

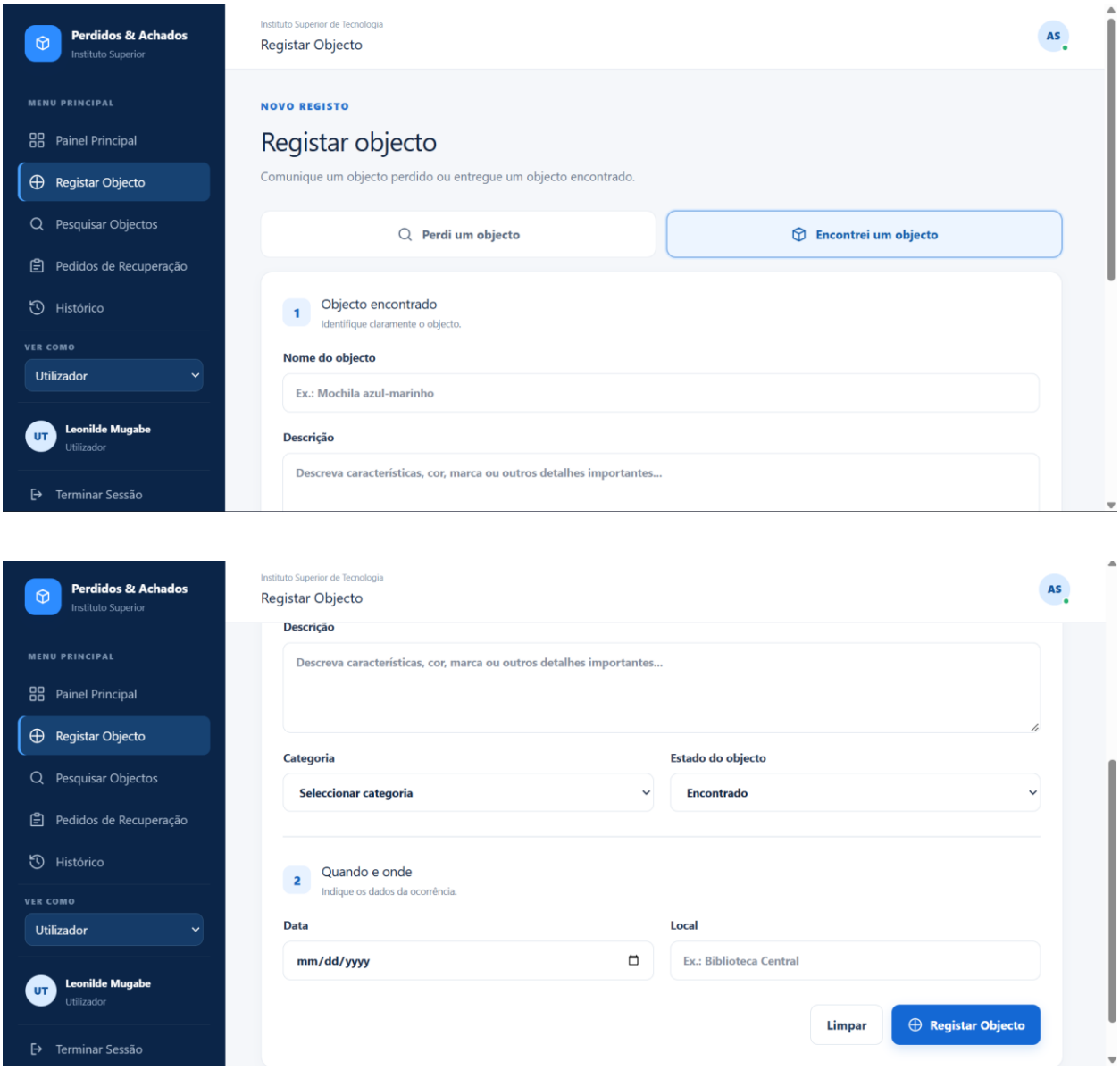


Figura 10: Pesquisa e consulta de objectos.

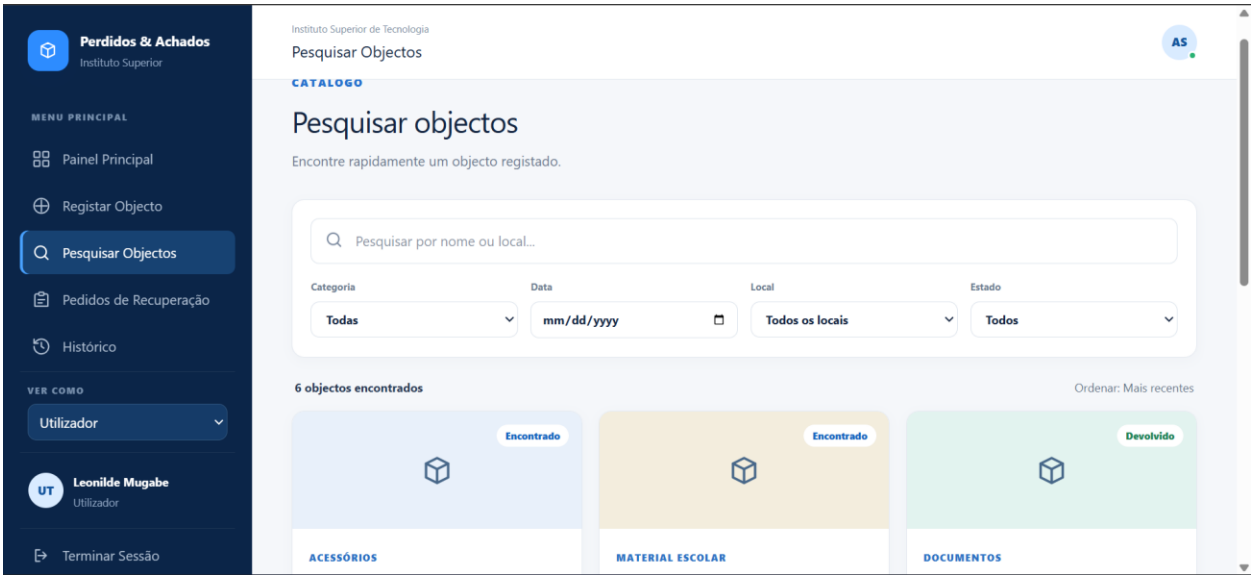


Figura 11: Hisórico de actividade



Interface do Responsável pelos achados

Figura 12: painel principal

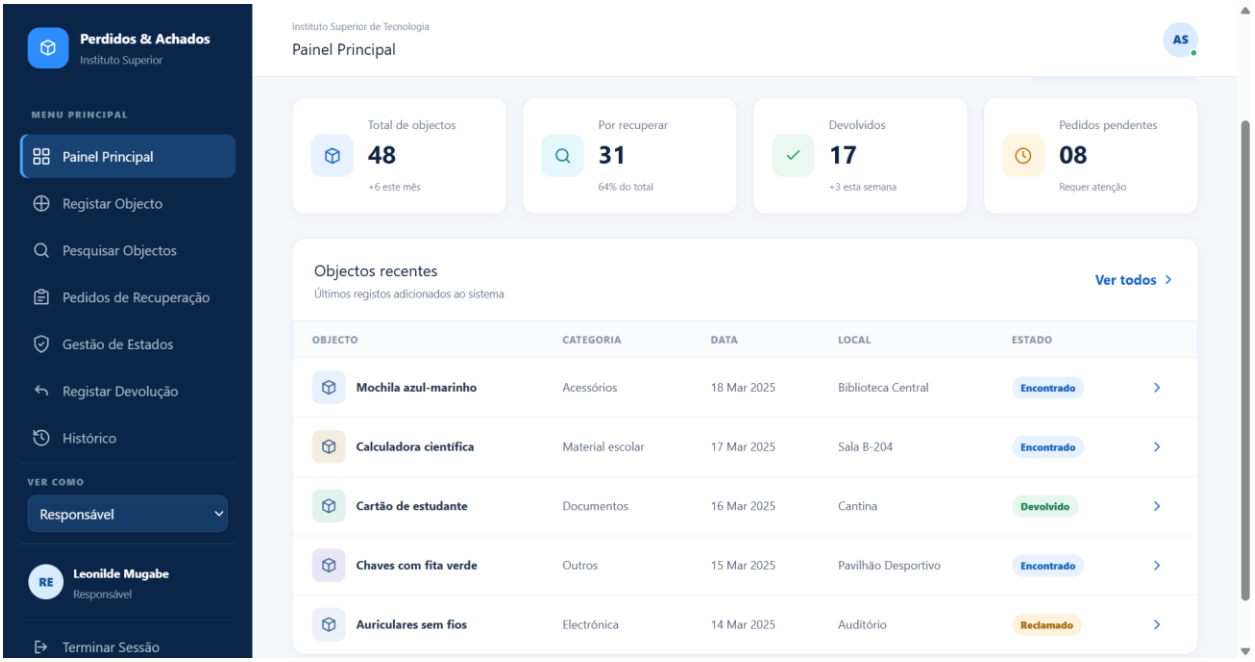
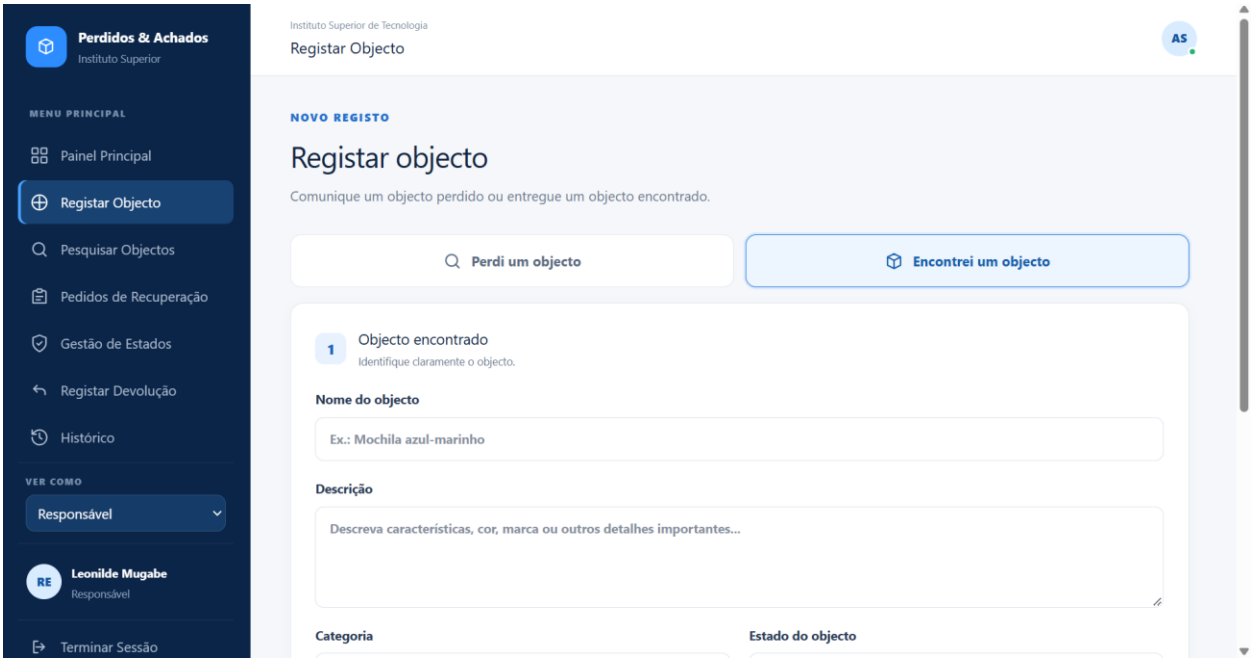


Figura 13: registo de objecto



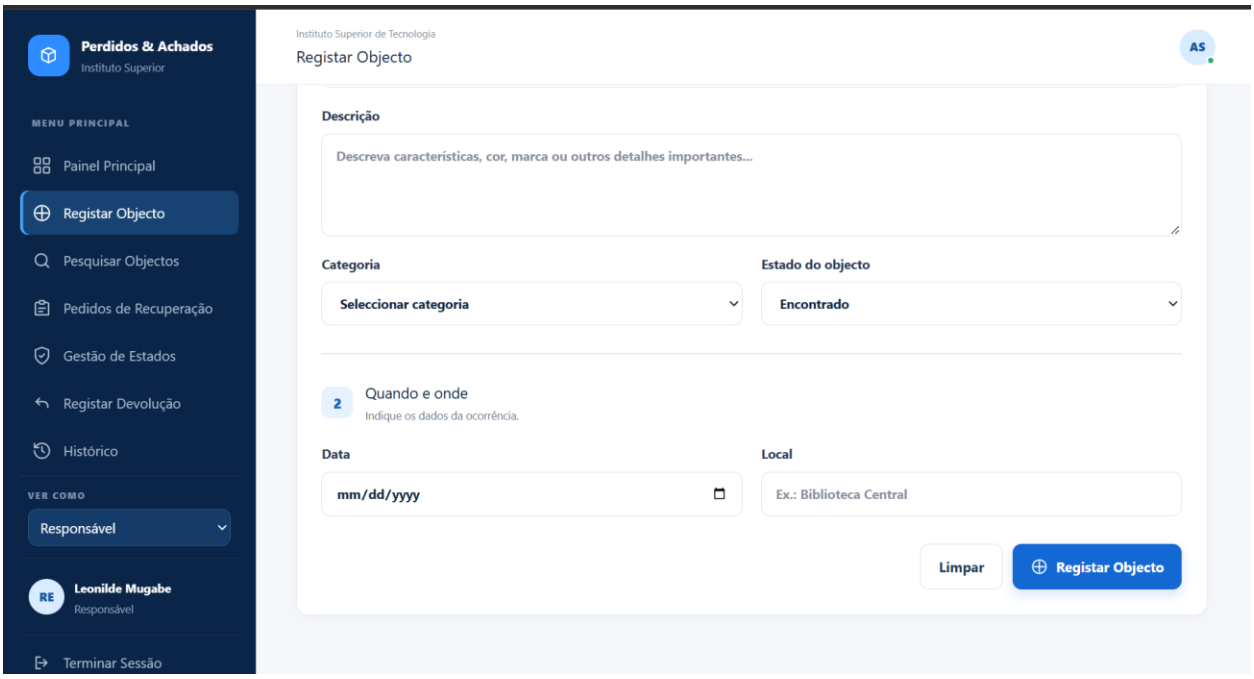


Figura 14: formulário para pesquisa de objectos

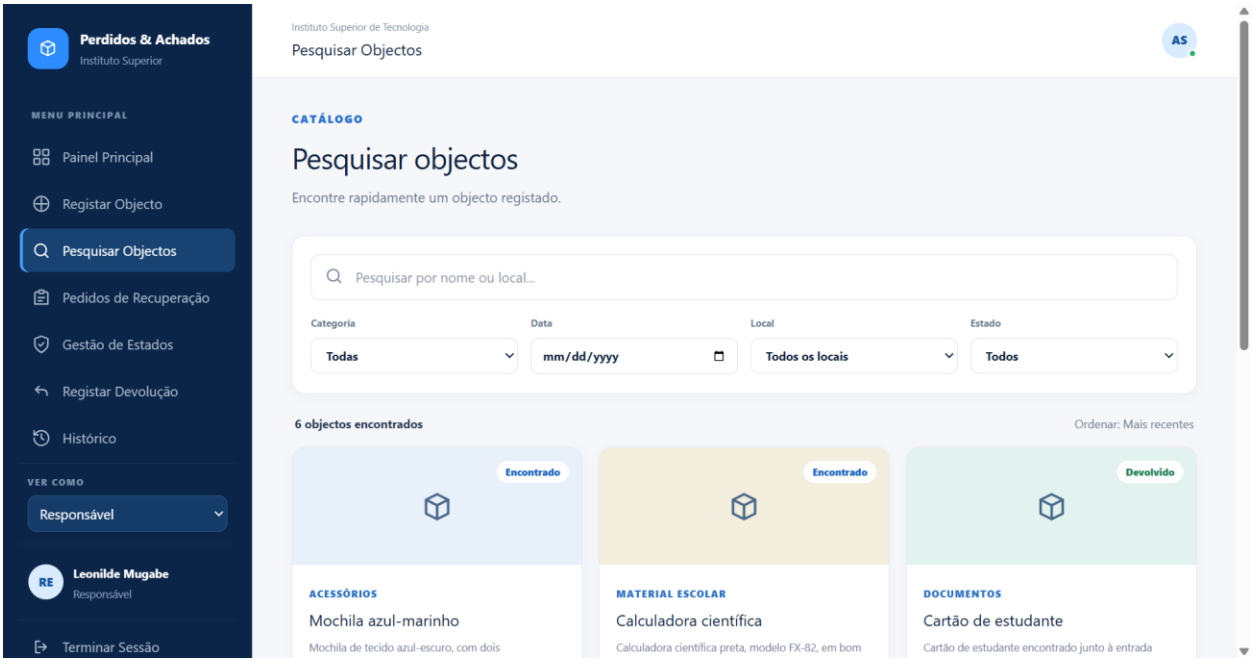


Figura: Pedidos de recuperação de objectos



Figura 15: Gestão de estados

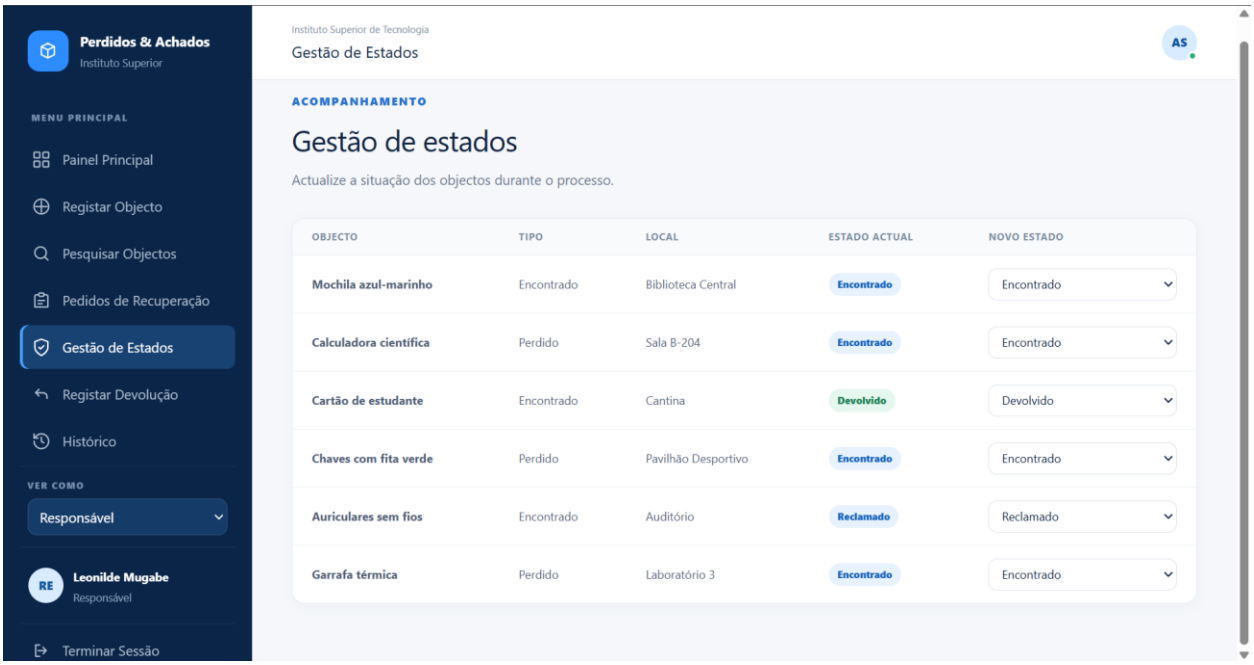


Figura 16: registo de devolução de objectos

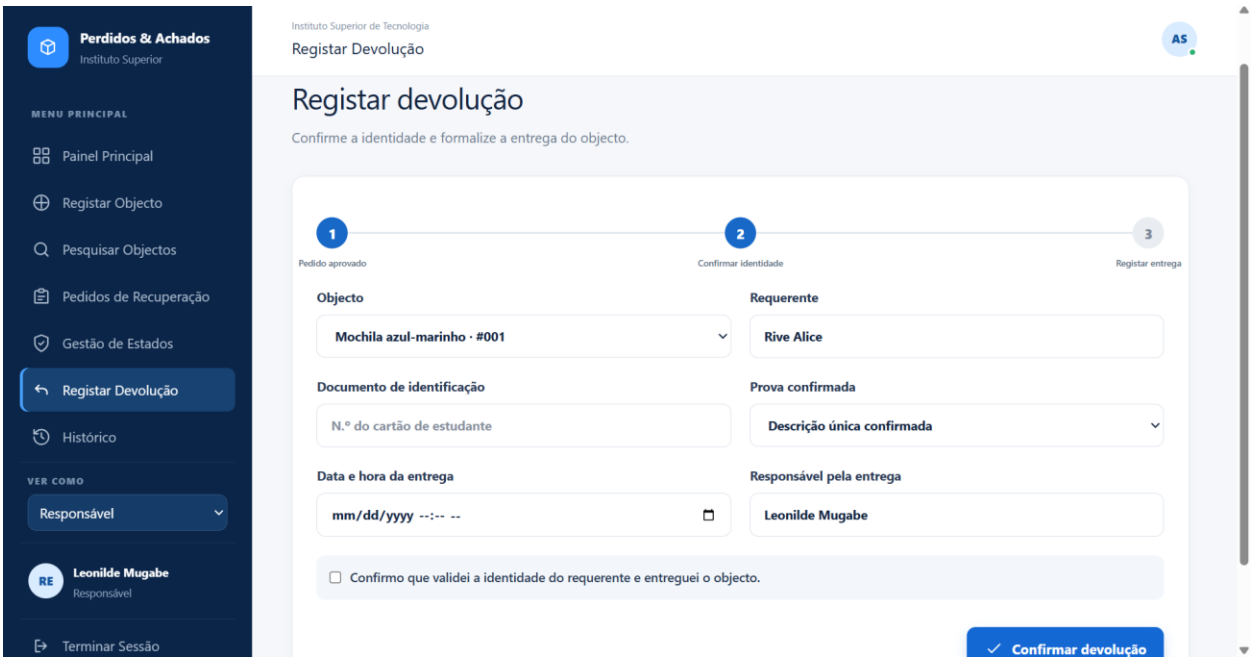
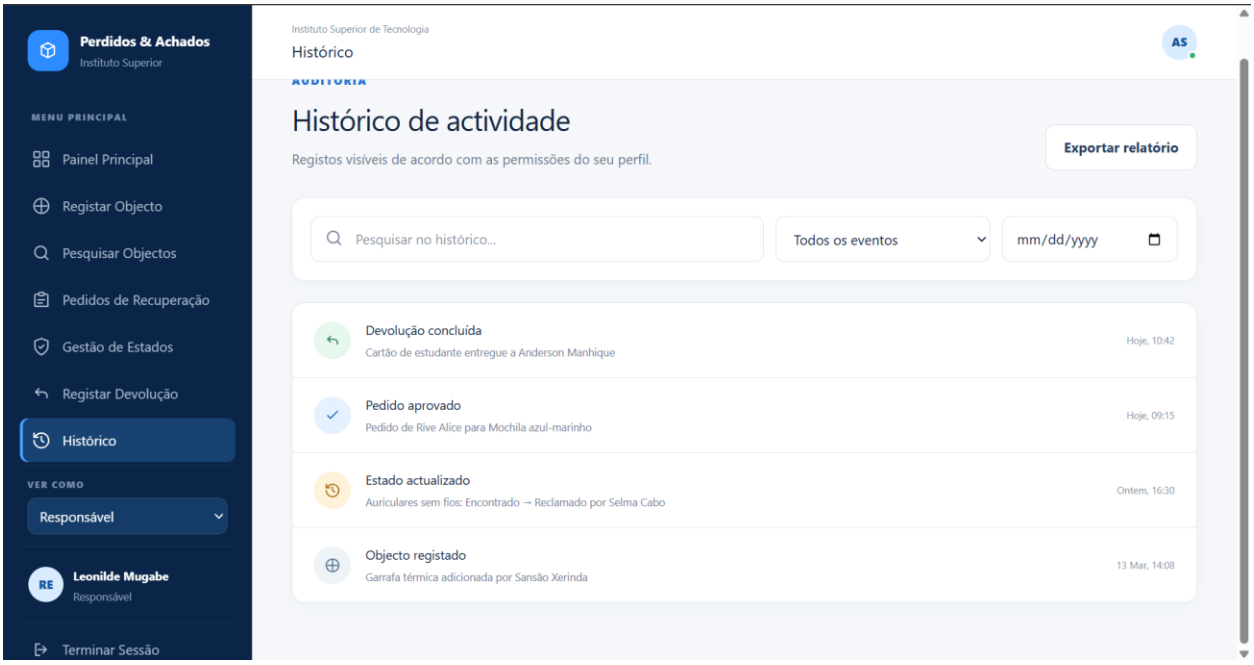


Figura 17: Historico da Actividade



5 CAPITULO V: PADRÕES DE PROJECTO

5.1 Padrões de projecto

Segundo Sommerville (2018), os padrões de projecto são soluções gerais e reutilizáveis para problemas comuns de concepção de software, especialmete em sistemas orientados a objectos. Estes padrões descrevem como as classes e os objectos se organizam, interagem e distribuem responsabilidades para resolver um problema específico num determinado contexto. Não representam soluções prontas a implementar, mas modelos que podem ser adaptados a diferentes situações de desenvolvimento de software.

Um padrão de projecto é constituido por quatro elementos fundamentais: o nome, o problema, a solução e as consequências. O nome permite identificar e comunicar o padrão; o problema descreve a situação em que pode ser aplicado; a solução apresenta os elementos envolvidos, as suas relações e responsabilidades; e as consequências permitem avaliar as vantagens, desvantagens, custos e beneficios da sua utilização.

No sistema de Gestão de Objectos Perdidos e Achados, foram seleccionados dois padrões de projrcto: Singleton e Model-View-Controller (MVC). A escolha destes padrões tem como tem como finalidade melhorar a aorganização da aplicação, facilitar o acesso aos recursos partilhados e separar as responsabilidades dos diferentes componenetes do sistema.

5.1.1 Padrão singleton

Problema que resolve

Durante o desenvolvimento de uma aplicação, podem existir recursos que devem ser partilhados por diferentes componentes do sistema. Um exemplo é a configuração da ligação à base de dados, que pode ser necessária durante o registo, a consulta e a actualização dos projectos.

Se cada componente criar e gerir a sua própria instância de configuração, poderá ocorrer repetição de código e tornar-se mais difícil manter uma configuração consistente.

Justificação da escolha

Escolheu-se o padrão Singleton para permitir o acesso centralizado a um recurso partilhado, como um gestor de configuração da ligação à base de dados. Esta abordagem pode contribuir para reduzir a duplicação de código e facilitar a manutenção do sistema.

Aplicação no projecto

No projecto de Gestão de Objectos Perdidos e Achados, o padrão Singleton poderá ser aplicado a uma classe responsável por disponibilizar a configuração necessária para o acesso à base de dados. Essa classe poderá ser utilizada pelos componentes responsáveis pelo registo de objectos, pela pesquisa e pela gestão dos pedidos de recuperação.

A aplicação deste padrão deverá ser feita de forma controlada, tendo em conta a gestão das ligações e a necessidade de evitar problemas quando vários utilizadores acedem ao sistema.

Vantagens

- Reduz a criação desnecessária de instâncias de uma classe
- Centraliza o acesso a um recurso partilhado.
- Pode diminuir a repetição de código relacionado com a configuração.
- Facilita a manutenção da configuração centralizada.

Desvantagens

- Pode criar dependências excessivas entre componentes
- Pode dificultar os testes unitários se a classe estiver demasiado ligada a outras partes do sistema.
- A utilização inadequada pode originar problemas de concorrência.
- Não é necessário para tdods os tipos de classes, devendo ser utilizado apenas quando existe uma justificação concreta.

5.1.2 Padrão Model-View-Controller (MVC)

Problemas que resolve

Num Sistema de software, a mistura das regras de negócio, dos dados e da apresentação das informações na mesma classe pode dificultar a compreensão e a manutenção do código.

No sistema proposto, existem diferentes operações, como registar objectos, pesquisar objectos encontrados, solicitar a recuperação e analisar pedidos. Se todas estas responsabilidade estiverem concentradas nos mesmos componentes, a introdução de alterações poderá torna-se mais complexa.

Justificação da escolha

Escolheu-se o padrão Model-View-Controller (MVC) porque permite separar a aplicação em três componentes principais: Model, View e Controller. Esta separação torna a estrutura do sistema mais organizada e facilita a manutenção, os testes e a introdução de novas funcionalidades.

Aplicação no projecto

No sistema de Gestão de Objectos Perdidos e Achados, o padrão MVC poderá ser aplicado da seguinte forma:

- **Model (Modelo):** representa os dados e as regras relacionadas com as entidades do sistema, como Utilizador, Objectos e PedidoRecuperacao.
- **View (Visão):** corresponde ás interfaces através das quais os utilizadores poderão registar, consultar e acompanhar informalões sobre os objectos e os pedidos.
- **Controller (Controlador):** recebe as acções dos utilizadores, coordena o processamento das operações e encaminha os resultados para a vista adequada.

Por exemplo quando um utilizador pretende registar um objeccto perdido, a view apresenta o formulário, o controller recebe os dados e coordena a operação, enquanto o modelo representa os dados e as regras aplicáveis ao registo.

Vantagens

- Separa a apresentação, os dados e o controlo das operações
- Facilita a manutenção e a orgaznização do código.
- Permite testar os componentes de forma mais independente.
- Facilita a introdução de novas funcionalidades.
- Reduz a dependência directa entre a interface e as regras de negócio.

Desvantagens

- Pode aumntar o número de classes e ficheiros do projecto.
- Exige uma organização inicila mais cuidadosa.
- Pode torna-se desnecessáriamente complexo em aplicações muito pequenas.
- Requer que as responsabilidades de cada compnente sejam bem definidas.

6 CAPITULO VI: ESPECIFICAÇÃO FORMAL

A especificação formal é uma descrição inequívoca do que o sistema deve fazer. As especificações formais são a maneira mais precisa de especificar sistemas e, portanto, de reduzir o espaço para mal entendidos. Muitos defesnsores dos métodos formais acreditam que vale a pena criar uma especificação formal, mesmo sem refinamento ou prova do programa. (Sommerville, 2018).

Operação 1: Registar um objecto encontrado

Esta operação permite que o responsável pelos objectos encontrados registe no sistema um objecto que foi localizado na instituição.

Pré – condições

- O responsável deve estar autenticado no sistema
- O responsável deve possuir permissões para registar objectos
- Os dados obrigatórios do objecto devem estar preenchidos
- O objecto não deve estar registado de froma duplicada, de acordo com os critérios definidos pelo sistema.

Estado inicial:

O responsável encontra-se autenticado e o sistema está disponível para receber os dados de um novo objecto encontrado. O objecto ainda não consta no registo correspondente.

Pós – condições

- O objecto fica registado na base de dados.
- O sistema atribui um identificador único ao objecto.
- O objecto fíca associado ao registo do responsável que efectuou a operação.

- O objecto fica disponível para consulta, de acordo com as regras de acesso definidas.

Estado final:

O objecto encontra-se registado no sistema com um identificador único e com os respecctivos dados guardados, podendo ser consultado pelos utilizadores autorizados.

Operação 2: Solicitar a recuperação de um objecto encontrado e registado no sistema, caso considere que lhe pertence.

Pré – condições

- O utilizador deve estar autenticado no sistema.
- O objecto deve existir no registo do sistema.
- O objecto não deve estar marcado como devolvido.
- O utilizador deve fornecer as informações necessárias para justificar o pedido de recuperação.

Estado incial:

O objectivo encontra-se registado no sistema e ainda não foi devolvido ao seu proprietário. O utilizador está auntenticado e pode iniciar um pedido de recuperação.

Pós – condições

- O sistema regista o pedido de recuperação
- O pedido fica associado ao utilizar que o submeteu
- O pedido fica associado ao objecto pretendido.
- O pedido assume o estado inicial de análise, aguardando a decisão do reponsável.

Estado final:

O pedido de recuperação encontra-se registado no sistema, associado ao utilizador e ao objecto correspondente, ficando disponível para análise pelo responsável.

7 CAPITULO VII: TESTES

7.1 Testes de software

Segundo Sommerville (2018), os testes de software têm como finalidade verificar se um programa corresponde ao que foi concebido para fazer e identificar defeitos antes de ser disponibilizado para utilização. Este processo consiste em executar o programa com dados de teste e analisar os resultados obtidos, de modo a detectar erros, comportamentos anormais e obter informações dobre as caracteristicas não funcionais do software.

7.2 Teste Unitário

O teste unitário consiste em testar individualmente os componentes de um programa, como métodos e classes, com o objectivo de verificar se funcionam correctamente. No caso de funções ou métodos individuais, os testes são realizados através da execução dessas rotinas com diferentes parâmetros de entrada, permitindo analisar os resultados obtidos e identificar possíveis erros. (Sommerville

7.3 Testes de integração

Segundo Figueiredo (s,d), os testes de integração têm como objectivo verificar se duas ou mais unidades de software funcionam corectamente em conjunto. Este tipo de teste consiste em avaliar grupos de unidades integradas para formar um sistema ou subsistema, dando especial atenção às interfaces de comunicação e às interacções entre os diferentes componentes. Geralmente, estes testes são da responsabiidade de um equipa independente de testes.

7.4 Aplicação dos testes

Como o sistema na fase actual está baseada num protótipo, foram apenas identificados os testes que serão feitos futuramente para o sistema.

8 CAPITULO VIII: QUALIDADE E MELHORIAS

8.1 Qualidade de Projecto

Qualidade de projeto refere-se às características que os projetistas especi cam para um produto. A qualidade dos materiais, as tolerâncias e as especi cações de desempenho, todos são fatores que contribuem para a qualidade de um projeto. (Sommerville, 2018)

Segundo Bueno e Campelo (s,d), a qualidade de projecto refere-se ao conjunto de caracteristicas e requisitos definidos pelos projectistas para garantir que um produto ou sistema corresponde às necessidades e expectativas estabelecidas. Estas caracteristicas podem incluir o desempenho, a tolerância, a fiabilidade, a segurança, a facilidade de utilização e a capacidade de manutençao do sistema, dependendo do tipo de produto e da sua finalidade.

O enfoque prinncipal da qualidade de pprojecto esta na definição dos requisitos, na elaboração das especificações e na concepção do sistema. Nesta fase, são estabelcidos os critérios que o produto deverá cumprir e as caracteristicas que permitirão avaliar a sua qualidade.

8.2 Métricas

Atributo de qualidade	Métrica	Critério de aceitação
Usabilidade	Percentagem de utilizadores que conseguem realizar as tarefas principais sem ajuda	pelo menos 80% dos participantes devem concluir as tarefas definidas sem ajuda
Segurança	Percentagem de cenários de acesso que respeitam os perfis e as permissões definidos	100% dos cenários de acesso avaliados devem respeitar as permissões previstas
Desempenho	Tempo de resposta de uma pesquisa de objectos	a pesquisa deve responder em até 3 segundos, nas condições de teste estabelecidas.

Integridade dos dados	Percetagem de casos de teste em que as regras de validação são respeitadas.	100% dos casos de teste definidos para as regras criticas devem apresentar o resultado esperado.
Manutenibilidade	Percentagem de componentes e elementos da interface organizados de acordo com as convenções definidas	Pelo menos 90% dos elementos avaliados devem respsitar as convenções de organização e consistência.

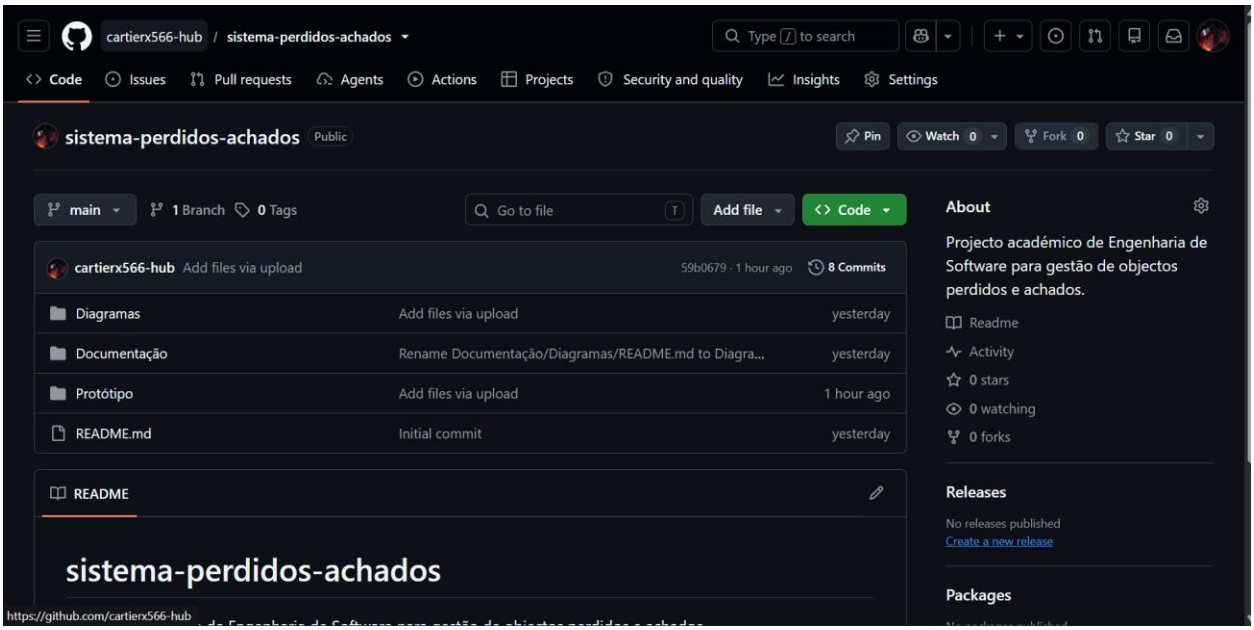
9 CAPITULO IX: GESTÃO DE VERSÕES

9.1 Controle de versões

Segundo sommerville (2018), o controlo de versões é uma prática de gestão da configuração de software que permite registar e acompanhar as alterações realizadas nos ficheiros de um projecto ao longo do tempo. Esta prática facilita a identificação das diferentes versões, a recuperação de versões anteriores quando surgem erros ou quando é necessário restaurar uma versão funcional.

O controle de versões também facilita o trabalho colaborativo, permitindo que vários programadores trabalhem no mesmo projecto e acompanhem as alterações realizadas por cada membro da equipa. Para isso, podem ser utilizadas ferramentas como Git, que ajudam a organizar o histórico das modificações e a gerir diferentes versões do código – fonte.

Repositório GitHub



Link do Repositório:

10 CAPITULO IX: CONCLUSÃO

A realização deste projecto permitiu propor uma solução informática para melhorar a gestão de objectos perdidos e achados numa instituição de ensino superior. Verificou-se que a falta de um sistema centralidado pode dificultar o registo, a pesquisa e a recuperação dos objectos. Para responder a este problema, foram definidos os requisitos de sistema, eleborados os diagramas UML e o modelo entidade – relacionamento, e desenvolvido um protótipo das interfaces.

Segundo Sommerville (2018), a modelação permite representar a estrutura e o comportamento de um sistema, facilitando a compreensão dos seus requisitos e funcionalidades. Por sua vez, Pressman e Maxin (2021), destacam a importância de avaliar a qualidade do sioftware durante o seu desenvolvimento , considerando aspectos como a funcionalidade, a usabilidade e a fiabilidade. Estes princípios foram considerados na concepção da solução proposta.

11 REFERÊNCIAS BIBLIOGRÁFICAS

Bezerra, E. (2014). Princípios de Análise e Projeto de Sistemas Com UML. https://eic.cefet-rj.br/papsuml3ed/papsuml3ed-caderno-zero.pdf

Bueno, C. de F. dos S., & Campelo, G. B. (s.d.). Qualidade de software. Departamento de Informática, Universidade Federal de Pernambuco. https://www.cin.ufpe.br/~mrsj/Qualidade/Qualidade%20de%20Software.pdf

Dias, R. (2019). O Modelo Incremental - Contexto Delimitado - Medium. https://medium.com/contexto-delimitado/o-modelo-incremental-b41fc06cac04

Figueiredo, E. (s.d.). Teste de integração e teste de sistema [Diapositivos de aula]. Departamento de Ciência da Computação, ICEX, Universidade Federal de Minas Gerais. https://homepages.dcc.ufmg.br/~figueiredo/disciplinas/aulas/testes-de-release_v01.pdf

Nunes, M., & O'Neill, H. (2003). Fundamental de UML (2.ª ed.). FCA — Editora de Informática.

Pablos, F. D. Y., & Feitosa, M. D. (2020). Acessibilidade em métodos ágeis: uma revisão sistemática da literatura. Research, Society and Development. https://www.researchgate.net/publication/340238406_Acessibilidade_em_metodos_ageis_uma_revisao_sistematica_da_literatura

Peters, J. F., & Pedrycz, W. (2001). Engenharia de Software: Teoria e Prática. (pp. 40-59) https://pdfslide.tips/documents/engenharia-de-software-teoria-e-prtica-james-e-peter-witold-pedrycz.html

Pressman, R., & Bruce R. M. (2002). Engenharia de Software: uma abordagem profissional. (5a ed.): AMGH.

Pressman, R. S. (2021). Engenharia de software – Uma Abordagem Profissional. (9a ed.) AMGH

Sommerville, I., & Graciano, N. (2018) - Engenharia de Software 10 ed.

Vieira, A. F., Lima, A. C. J. S., & Carr, C. N. (2023). Melhorando a efetividade de projetos de desenvolvimento de software através de uma abordagem híbrida: Uma análise de resultados e impactos. Research, Society and Development. https://rsdjournal.org/index.php/rsd/article/view/40693

Wazlawick, R. S (2013). Engenharia de software: conceitos e práticas

WordPress. (2019). Diagrama UML. [Material de apoio/apontamentos]. https://youbestage.wordpress.com/wp-content/uploads/2019/05/diagramas-uml-1-1.pdf